

13.2 Self-Consistency and Tree-of-Thoughts

Majority vote +17.9 pp on GSM8K. Vote on answers, not wording.

These notes match the lecture slides. Use **(slides)** on the course hub for the deck.

One chain is a sample, not a proof. **Self-consistency** asks for several traces and **votes**. **Tree-of-thoughts** (ToT) searches among partial plans instead of committing to a single left-to-right story. By the end you should be able to run a majority vote on parsed answers, say why temperature 0 is not self-consistency, and contrast a tree with a tool loop.

1. What self-consistency and Tree-of-Thoughts are

Self-consistency (SC) is majority vote over several CoT traces. Sample k traces (temperature > 0), parse an answer from each, and take the majority. The hope: wrong paths disagree; right paths land on the same number. Voting on the **answer**, not on the wording of the steps, is the usual rule.

Tree-of-Thoughts (ToT) is **search** among partial thoughts, not i.i.d. samples. A “thought” is a node: a subgoal, a candidate equation, a next move in a game. You expand a few children, **score** them (a prompt, a heuristic, or a checker), and keep a beam or run BFS/DFS. Greedy CoT is a tree of depth equal to one chain and width 1. ToT buys width.

Temperature 0, $k = 3$ is **not** SC: you will get identical greedy traces. Self-consistency does not fix a shared bug. If every trace uses the same wrong formula, the majority is still wrong. Wang et al. reported that majority vote raised GSM8K by **+17.9 percentage points** versus greedy CoT. That is a lift on the **box**. It is not a faithfulness claim (note **13.3**).

This course’s version of extra test-time compute is k traces. Do **not** assign DeepSeek-R1, GRPO, or a 256-expert run as homework.

2. Why we use it

One CoT is noisy. A single greedy chain can hit a digit bug and box 32 for $23 + 19$. Sampling several traces and voting on the parsed number is a cheap noise reducer when errors are **diverse**.

ToT helps when you need a tree, not k independent tapes. For $6 \times 7 + 5$, nodes “do multiply first” versus “do add first” can be scored before you decode a full wrong chain. ReAct (note **14.2**) is a different control loop: the node can be a **tool call**, not only more text. Vote versus tree versus tools are three knobs. Do not mix the names in the report.

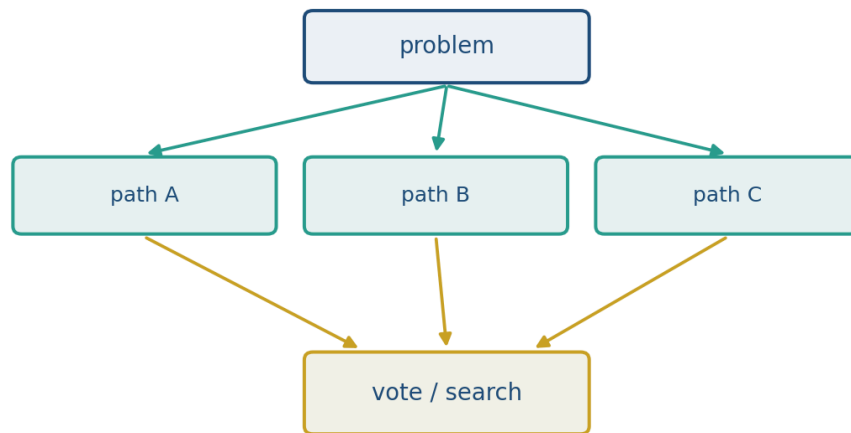
Always include a greedy CoT baseline at $k = 1$. Cost belongs in the table: tokens, or wall time on the hardware you used.

3. Architecture

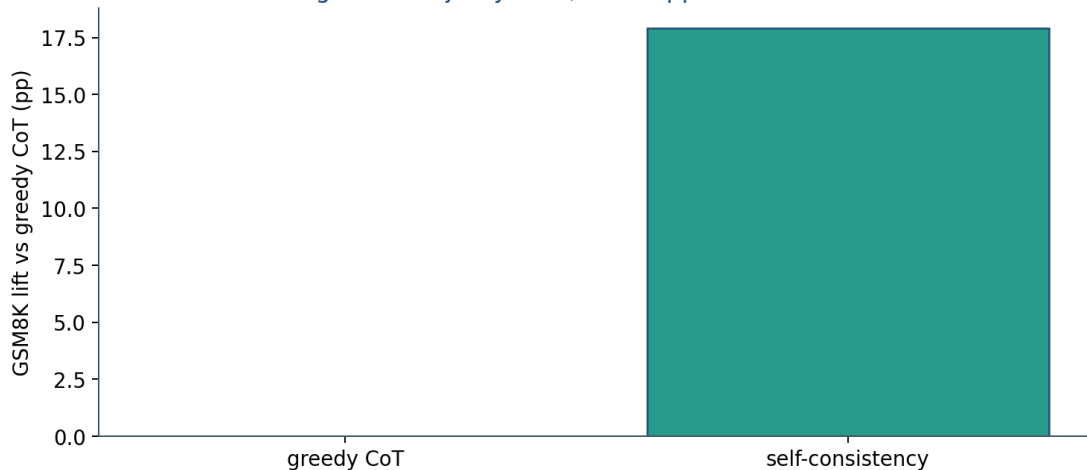
Self-consistency is a decoder plus a parser plus a vote. There is no new network. You pay k forwards. Report k and the parser (last integer, boxed span, regex). Lab 13 does that vote on constructed strings.

ToT is a frontier of partial strings, a scorer (LM or checker), and a search rule (beam / DFS / BFS). That is search with an LM as the proposal distribution. ToT has no single loss.

Self-consistency votes; Tree-of-Thoughts searches partial steps



CS224N L12: Wang et al. majority vote, +17.9 pp on GSM8K.



A cheap ToT scorer you can actually run in this course: last integer equals a known subgoal, a unit test, or `calc` from Week 14. An unconstrained “does this thought look good?” prompt is another noisy LM, not a checker.

4. How it works, step by step

Self-consistency.

1. Use a CoT prompt from note **13.1**. Set temperature > 0 so traces can disagree.

2. Sample k independent traces.
3. Parse one answer from each (last integer, boxed span, or a regex). Do not vote on full step strings: different wording of the same algebra would split the vote.
4. Take the majority. Write down a tie-break rule in advance.
5. Plot accuracy against k if you use this in a project; the curve usually flattens.

Tree-of-Thoughts.

1. Represent a partial plan as a node (for example “multiply first” versus “add first”).
2. Expand a few children from the frontier.
3. **Score** them with a checker when you have one; prune the rest.
4. Decode a full chain only along surviving branches.

Lab 13’s vote is the SC half: three constructed traces for $23 + 19$, majority on the parsed number. The ToT cartoon in the worked example is search, not three i.i.d. tapes.

5. Mathematical formulas

Self-consistency parses each trace τ_i to an answer and takes the mode:

$$\hat{y} = \text{mode}\{\text{parse}(\tau_1), \dots, \text{parse}(\tau_k)\}.$$

ToT has no single loss; it is search. Accuracy versus k is the self-consistency plot. For ToT, accuracy versus expansion budget (nodes scored). Always include greedy CoT at $k = 1$.

6. Positive points and negative points

Positive.

- SC is simple: sample, parse, vote. Lab 13 can do it on constructed strings.
- Diverse errors cancel; a $k = 1$ miss can become a majority hit.
- The GSM8K-style lift is a real box-accuracy result you can cite as motivation, not as a faithfulness claim.
- ToT can kill a wrong branch before you pay for a full chain, if the scorer is a checker.

Negative.

- Cost scales with k (or with nodes scored).
- Shared bugs survive the vote: three copies of $23 + 19 = 32$ still vote 32.
- Temperature 0 produces identical traces, so $k > 1$ is wasted.

- ToT needs a scorer; an unconstrained “looks good?” prompt is not a checker. The search math is not in this hour.
- ReAct (note **14.2**) calls tools instead of searching thoughts. Mixing the names hides the knobs.

When not to. A lookup that does not benefit from extra traces. A task where you already have a calculator: call the tool (Week 14) instead of voting on invented arithmetic.

7. What to measure

Accuracy versus k is the self-consistency plot. For ToT, accuracy versus expansion budget (nodes scored). Always include a greedy CoT baseline at $k = 1$. Cost belongs in the table: tokens, or wall time on the hardware you used.

8. Teaching this note

About **35 minutes** at the board, then **~10 minutes** of video. Self-consistency is the vote; the ToT paper is the search cartoon.

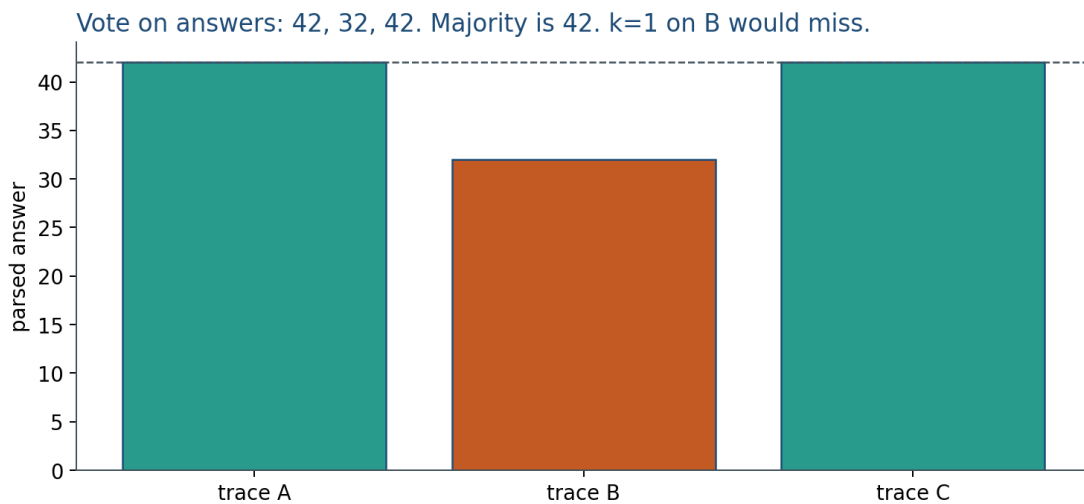
- **0–12 min.** Temperature > 0 , k traces, vote on the parsed answer. Tie-break rule.
 - **12–22 min.** ToT as width: expand, score, keep a beam. Contrast with ReAct (tools).
 - **22–33 min.** Worked majority vote on three traces (Lab 13 numbers).
 - **Then** play Karpathy Deep Dive **1:46:56–2:01:11** (models need tokens to think). Pause on distributing hard work across tokens; then say: self-consistency is k of those traces, ToT is a tree of them.
-

9. Worked example

Problem: $23 + 19$, gold 42. Three traces, answers only (Lab 13 style):

trace	parsed answer	steps (sketch)
A	42	$20 + 19 = 39$, $39 + 3 = 42$
B	32	$23 + 19 = 32$ (digit bug)
C	42	carry arithmetic to 42

Vote on **answers**: 42, 32, 42. Majority is 42 (2 of 3). $k = 1$ using only B would have scored 32 and missed. Self-consistency caught the disagreement.



Tie example: answers 42, 32, 32. Majority is 32, which is **wrong**. Voting is not a proof; it is a noise reducer when errors are **diverse**.

Shared-bug example: all three use $23 + 19 = 32$. Vote is 32. $k = 3$ does not help. A checker (note **13.3**, or a calc tool in Week 14) would.

ToT cartoon for $6 \times 7 + 5$: nodes “do multiply first” vs “do add first.” A cheap scorer (PEMDAS, or `eval`) kills the add-first branch before you decode a full wrong chain.

10. Where students get stuck

- Sampling at temperature 0 and wondering why all k traces match. You need diversity for a vote.
- Voting on the **wording** of the steps. Parse an answer; majority on that.
- Calling ToT “just CoT with more words.” ToT scores **partial** nodes and prunes.

11. Video

Watch [Andrej Karpathy, Deep Dive into LLMs like ChatGPT](#), **1:46:56–2:01:11**.

Pause on “models need tokens to think”: extra tokens are extra compute. This course’s self-consistency is k independent traces plus a vote; ToT is search over partial thoughts. Optional: **2:14:42–2:27:47** (RL that explores many solutions) if you want a later-week bridge. Note **13.3** uses a different chapter of the same URL.

12. Practice

1. Why does self-consistency need temperature greater than 0?
2. Give one failure that majority vote cannot catch.

3. ToT scores partial thoughts. Why is a cheap checker (arithmetic, a unit test) a better scorer than another unconstrained prompt, when you have one?
4. Traces vote 47, 65, 47, 47. What is the majority, and what is k ? Gold is 47. Did vote beat a first-trace-only policy if the first trace was 65?
5. Answers 408, 408, 238 for 17×24 . Write the vote. If you instead vote on full step strings (all different wording), what goes wrong?